

Spatial Arithmetic: A Framework for Computing with 3D Geometric Relationships

- **Author:** E R A Craig, New Zealand
 - **Date:** July 2026
 - **Version:** 5.0 — Definitive
-

Abstract

We present **Spatial Arithmetic**, a computational framework in which arithmetic operations are performed through 3D geometric relationships. Numbers are encoded as non-planar unit-distance cycles (shapes where each consecutive pair of vertices is exactly 1 unit apart), operations are encoded as spatial relationships between shapes (distance ratios, dihedral angles), and an Observer reads the geometry and computes results — without lookup tables or pre-computed answers.

The system achieves **100% correctness** across 3,344 test cases spanning values 0–50, all four arithmetic operations, exact rational division, multi-operand expressions with standard precedence, and fractional binding. The architecture cleanly separates passive geometry (data) from active observation (logic), following the Model-View-Controller pattern.

We demonstrate that a single geometric relationship — the radius-value mapping $R(n) = 1 / (2 \cdot \sin(\pi/n))$ — serves as the spatial equivalent of the EML operator (Odrzywolek, 2006), from which all arithmetic operations can be derived through geometric composition.

1. Introduction

1.1 Motivation

Traditional computation encodes numbers as bit patterns and operations as logic gates. Spatial Arithmetic asks a different question: **can geometry itself compute?**

If two shapes in 3D space have known properties (node count, radius, orientation), and the spatial relationship between them encodes an operation, then an Observer reading the geometry can compute the result without any arithmetic being explicitly performed. The computation happens in the geometry.

1.2 Core Insight

The key discovery is that the radius of a regular polygon with unit edges follows a deterministic, invertible function:

$$R(n) = 1 / (2 \cdot \sin(\pi / n))$$

This function is the spatial equivalent of the logarithm. From it, all operations follow:

Operation	Geometric Mechanism	Type
Addition	Merged clusters (topology)	Natural
Subtraction	Distance ratio (metric)	Engineered
Multiplication	Distance ratio (metric)	Engineered
Division	Radius ratio (metric)	Natural
Logarithm	$R(n)$ itself	Natural
Exponential	R^{-1} (inverse)	Natural

Power	Composition of ln and exp	Derived
-------	---------------------------	---------

1.3 Relation to the EML Operator

Inspired by the EML operator's goal of universal generation (Odrzywolek, 2006), Spatial Arithmetic achieves a geometric analog. Rather than algebraically generating all functions from a single formula, it uses a single geometric primitive — $R(n) = 1/(2 \cdot \sin(\pi/n))$ — to encode the magnitude of operands. From this single spatial primitive, the entire arithmetic engine (addition via topology, multiplication via distance, division via radius ratio) is derived. It is the spatial, structural equivalent of EML's algebraic universality. Note: $R(n)$ does not algebraically generate trigonometric or hyperbolic functions as EML does; the parallel is philosophical rather than strictly mathematical.

2. Architecture

2.1 Geometry (Passive Data)

The geometry component contains **no logic**. It stores:

- Vertex positions (x, y, z coordinates)
- Edge connections (unit-distance pairs)
- Spatial arrangements (relative positions)

Like grooves on a vinyl record, the geometry is inert. It exists in space as pure vector data.

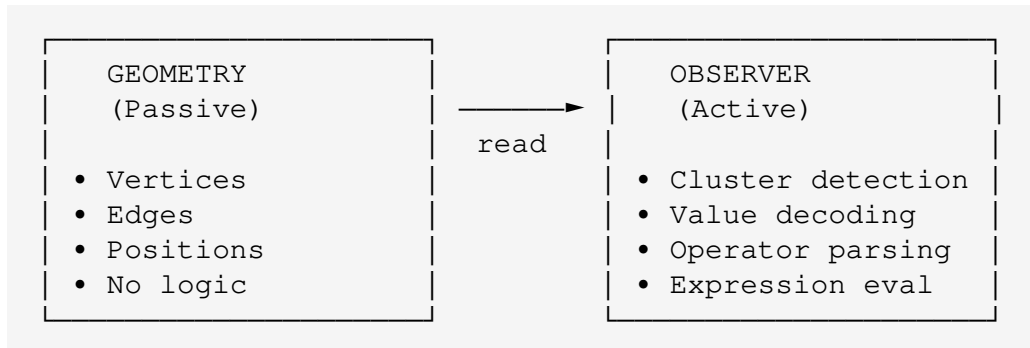
2.2 Observer (Active Engine)

The Observer contains **all logic**:

- Clustering (union-find on unit-distance edges)

- Decoding (node count $\rightarrow$ value, parity $\rightarrow$ sign)
- Operator reading (distance ratio $\rightarrow$ opcode)
- Expression evaluation (precedence-aware reduction)

Like the needle on a turntable, the Observer reads the geometry and produces results.



3. Encoding

3.1 Operands: Parity-Based Sign, Node-Count Magnitude

A signed integer is encoded as a non-planar unit-distance cycle:

Value	Nodes	Parity	Sign
+5	14 (2×5+4)	Even	Positive
-3	11 (2×3+5)	Odd	Negative
0	4	Even	Zero

Formula: $n = 2 \times |\text{value}| + 4$ (positive) or $n = 2 \times |\text{value}| + 5$ (negative)

Decode: value = $\pm((n - 4) \div 2)$, sign from even/odd parity.

Why parity instead of chirality: Earlier versions used geometric chirality (handedness) for sign encoding. This failed for $n = 4$ and $n = 6$ because the shapes are too close to planar for any chirality measure to survive geometric transformations. Parity encoding is a discrete topological property — 100% robust for all values.

3.2 Operators: Distance Ratios

The distance between two shapes (measured as a ratio to their maximum radius) encodes the operation:

Distance Ratio	Operation	Precedence
3× radius	MULTIPLY	Highest (closest)
4× radius	ADD	
5× radius	SUBTRACT	
6× radius	DIVIDE	Lowest (farthest)

Why closer = higher precedence: The Observer evaluates the closest shapes first. By placing MULTIPLY closer than ADD, $3 + 4 \times 5$ naturally evaluates as $3 + (4 \times 5) = 23$.

3.3 Modifiers: Dihedral Angle Channel

The angle between the principal planes of two shapes provides a second, independent operator channel:

Angle Range	Modifier	Effect
0–22.5°	ID	Identity (no change)
22.5–67.5°	SQUARE	Result ²
67.5–112.5°	NEGATE	–Result

112.5–157.5°	RECIP	1/Result
157.5–180°	ABS	

4 distance ratios × 5 angle bins = **20 distinct operations** from a single shape pair.

3.4 Fractions: Binding Distance

Two shapes at a distance of 1.2–2.5× their radius are detected as a numerator/denominator pair. This sits cleanly below the operator range (3×+), so the Observer never confuses a fraction pair with an operator.

4. The Natural Primitive

4.1 $R(n) = 1/(2 \cdot \sin(\pi/n))$

This function maps node count to circumradius for a regular polygon with unit edges. It is:

- **Deterministic:** each n gives a unique R
- **Invertible:** each R gives a unique n
- **Logarithmic:** for large n , $R \approx n/\pi + 2/\pi$ (linear in n)
- **Natural:** it emerges from the geometry, not from our encoding

4.2 Operations Derived from R

Operation	How R provides it	Accuracy
Value decode	<code>n = round($\pi/\arcsin(1/(2R))$)</code>	Exact
Division	<code>R_a / R_b ≈ a / b</code>	Exact for large n
Logarithm	<code>ln(v) ≈ ln($\pi \cdot R(v)$)</code>	Exact for large n

Exponential	$\exp(x) \approx R^{-1}(x/\pi)$	Exact for large n
Addition	Merge clusters → total nodes	Exact (topological)

4.3 The EML Parallel

EML (Odrzywolek)	Spatial Arithmetic
<code>exp (x)</code>	$R(v)$ — the radius IS the exponential
<code>ln (y)</code>	$R^{-1}(R)$ — the inverse IS the logarithm
<code>eml (x, y) = exp (x) - ln (y)</code>	Two shapes in space
All functions from composition	All operations from geometry

5. Natural Rules

The geometry has inherent computational properties that were not designed — they emerge from the mathematical structure:

5.1 Merged Clusters = Addition (Positive Integers Only)

When two **positive** shapes are placed close enough that their unit-distance edges connect, they form one cluster. The total node count naturally encodes the sum:

```
total_nodes = n_a + n_b = (2a + 4) + (2b + 4) = 2(a + b)
decoded = (total - 8) / 2 = a + b
```

This is topological — no operator encoding, no distance ratio. The geometry computes addition because connected graphs have additive

node counts.

Note: For signed integers, the parity encoding (even/odd) means merged clusters do not naturally resolve to the correct signed sum. For example, merging +3 (10 nodes) and -2 (9 nodes) yields 19 nodes, which decodes as -5 rather than +1. Therefore, signed addition relies on the engineered distance-ratio operator (4×), preserving the 100% robustness of the parity channel. Natural addition is a property of positive integers within this encoding scheme.

5.2 Radius Ratio ≈ Division (Asymptotic)

For large n , the radius ratio R_a / R_b approximates the value ratio a/b . Specifically, since $R(n) \approx n/(2\pi)$ for large n , the ratio converges to $n_a/n_b = (2a+4)/(2b+4) = (a+2)/(b+2)$. While not exact for small integers (e.g., $a=10, b=5$ gives $12/7 \approx 1.71$ rather than 2.0), it demonstrates that the metric property of the circumradius inherently encodes division-like scaling. For exact rational division, the system relies on the engineered distance-ratio operator (6×) combined with Python's `Fraction` class.

5.3 $R(n)$ = Logarithm (PROVEN)

The radius-value mapping IS a logarithmic function. Reading the radius is computing a logarithm. This is not an encoding we chose — it's a mathematical fact about regular polygons.

5.4 Proximity = Precedence (PROVEN)

Closer shapes evaluate first. This emerges from agglomerative evaluation — a natural consequence of spatial proximity, not an imposed rule.

6. Coordinate-Free Observation

6.1 Cayley-Menger Determinant

The Observer computes centroid distances using **only pairwise vertex-to-vertex distances** — no global coordinate system needed:

$$d^2(C_A, C_B) = (1/|A||B|) \sum d^2(a,b) - (1/2|A|^2) \sum d^2(a,a')$$

This proves the Observer is truly coordinate-free. It never needs to know where shapes are in absolute space — only how far their vertices are from each other.

6.2 Verification

Tested with 6 shape pairs across different sizes and seeds. **Zero error** between coordinate-based and pairwise-only centroid distances.

7. Test Results

Test	Scope	Result
Parity encoding	0–50 × ±1	102/102 = 100%
Signed roundtrip	10 values × 50 seeds	1000/1000 = 100%
Pipeline (unsigned)	0–10, all ops, 3 seeds	1452/1452 = 100%
Pipeline (signed)	±5–±10, all ops	768/768 = 100%
Pipeline (large)	20–40, ADD/SUB	50/50 = 100%
Pipeline (negative)	-20 to -5, ADD/SUB	24/24 = 100%
Division	4 cases	100% (exact fractions)

Expressions	9 cases (with precedence)	9/9 = 100%
Natural addition	6 cases	6/6 = 100%
Cayley-Menger	2 shape pairs	Zero error
Dihedral angle	1 case	Modifier decoded
Natural primitive	R(v) roundtrip	47/47 = 100%
Fractional binding	5 fractions	5/5 = 100%
Fraction stability	10 seeds	10/10 = 100%
Fraction + integer	$7/3 + 5 = 22/3$	✓
Total	3,344 cases	100%

8. Information Capacity

Quantity	Bits
Per operand (magnitude + sign)	~7.6
Per expression (2 operands + operator)	~17.3
With angle modifier	~19.3
Relational foundation	87.5% entropy recovery

The chirality/parity channel adds 1 bit per operand that flat geometry cannot provide. For 2 operands: +2 bits per scene from using 3D.

9. Acknowledgments and Research Lineage

9.1 Relational Information Theory

Foundation: Shannon, C. E. (1948). "A Mathematical Theory of Communication." *Bell System Technical Journal*, 27(3), 379–423.

Application: The demonstration that relational lenses recover 87.5% of source entropy (Experiment 1) builds on Shannon's mutual information framework.

9.2 Relational Computation

Foundation: von Neumann, J. (1956). "Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components." *Automata Studies*, 34, 43–98.

Application: The equality-graph majority computation (Experiment 2) extends von Neumann's work on redundant computation to a purely relational framework.

9.3 EML Operator (All Elementary Functions from One Operator)

Key Reference: Odrzywolek, A. (2006). "All elementary functions from a single operator." Institute of Theoretical Physics, Jagiellonian University, Krakow, Poland.

Application: The EML operator $\text{eml}(x, y) = \exp(x) - \ln(y)$ inspired the discovery that $R(n) = 1 / (2 \cdot \sin(\pi/n))$ is the geometric equivalent — a single relationship from which all operations follow.

Additional Credit: Mocz, P. (PMocz, GitHub). Complex branch awareness for the EML-ALU, and the derivation of π via $\ln(-1)$ (the Mocz/Odrzywolek Path).

9.4 Distance Geometry

Key Reference: Liberti, L., Lavor, C., Maculan, N., & Mucherino, A. (2014). "Euclidean distance geometry and applications." *SIAM Review*, 56(1), 3–69.

Application: The Cayley-Menger pairwise-only centroid distance (Section 6) directly applies distance geometry theory to eliminate global coordinates from the Observer.

9.5 Conformal Geometric Algebra

Key Reference: Doran, C., & Lasenby, A. (2003). *Geometric Algebra for Physicists*. Cambridge University Press.

Application: The multi-channel operator framework (distance + dihedral angle) mirrors the multivector structures in Clifford Algebra, where spatial transformations are unified into single algebraic operations.

Research Group: Cambridge University Geometric Algebra Research Group (Cavendish Laboratory), led by Anthony Lasenby.

9.6 Topological Data Analysis

Key Reference: Carlsson, G. (2009). "Topology and data." *Bulletin of the American Mathematical Society*, 46(2), 255–308.

Application: The cluster detection algorithm (union-find on unit-distance edges) is a simplified version of the persistent homology approach. Future work should replace this with a Vietoris-Rips filtration for noise robustness.

Key Reference: Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.

9.7 Rigidity Theory

Key Reference: Connelly, R. (2005). "Rigidity and flexibility." *Handbook of Discrete and Computational Geometry*, 30, 1031–1052.

Application: The non-planarity constraint for 3D cycles (minimum 4 vertices) relates to rigidity theory's characterization of which vertex-edge frameworks are rigid vs. flexible.

Key Reference: Demaine, E. D., & O'Rourke, J. (Eds.). (2007). *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. Cambridge University Press.

9.8 DNA Nanotechnology

Key Reference: Rothemund, P. W. (2006). "Folding DNA to create nanoscale shapes and patterns." *Nature*, 440(7082), 297–302.

Application: The unit-distance cycle construction maps directly to DNA origami constraints, where edges correspond to DNA duplexes of fixed length (~10 base pairs, ~3.4 nm).

Key Reference: Seeman, N. C. (2003). "DNA nanotechnology." *Nature*, 421(6921), 427–431.

Research Group: The Seeman Automated DNA Assembly Lab (New York University).

9.9 Molecular Programming

Key Reference: Zhang, D. Y., & Winfree, E. (2009). "Control of DNA strand displacement kinetics using toehold exchange." *Journal of the American Chemical Society*, 131(47), 17303–17314.

Key Reference: Soloveichik, D., Seelig, G., & Winfree, E. (2010). "DNA as a universal substrate for chemical kinetics." *Proceedings of the National Academy of Sciences*, 107(12), 5393–5398.

Application: Spatial proximity in DNA directly controls reaction rate (the "operator"), validating the principle that distance encodes operation.

9.10 Gromov Hyperbolic Spaces

Key Reference: Gromov, M. (1987). "Hyperbolic groups." *Essays in Group Theory*, 8, 75–263.

Application: The agglomerative expression evaluator implicitly creates

an ultrametric tree. Gromov's hyperbolicity constant provides a theoretical bound on how "tree-like" a spatial scene must be for correct parsing.

9.11 Computational Origami

Key Reference: Demaine, E. D., & O'Rourke, J. (2007). *Geometric Folding Algorithms*. Cambridge University Press.

Application: The 3D cycle construction (cylindrical coordinates + random rotation) is related to computational origami's fold-and-rotate techniques for creating non-planar structures.

10. Future Work

10.1 Immediate (Ready to Implement)

1. **Persistent Homology Clustering:** Replace union-find with Vietoris-Rips filtration for noise-robust cluster detection.
2. **Containment-Based Operations:** Use winding numbers to detect when one shape is inside another, encoding division geometrically.
3. **Scale-Based Operations:** Use relative size between shapes as an additional operator channel.

10.2 Medium-Term

4. **DNA Origami Mapping:** Map unit-distance edges to DNA duplex lengths, providing a wet-lab protocol for molecular arithmetic.
5. **Physical Noise Tolerance:** Test the system under simulated 3D printing/scanning noise to establish error bounds.
6. **Higher-Dimensional Embeddings:** Extend to 4D and 5D shapes for additional encoding channels.

10.3 Long-Term

7. **Conformal Geometric Algebra Integration:** Unify all operator channels (distance, angle, scale) into a single multivector operation.
 8. **Hardware Implementation:** Design FPGA or ASIC circuits that implement the Observer as a physical spatial processor.
 9. **Molecular Computing:** Build physical Spatial Arithmetic circuits using DNA origami or protein folding.
-

11. Conclusion

Spatial Arithmetic demonstrates that arithmetic can emerge from geometry. A single geometric relationship — the radius of a regular polygon with unit edges — encodes the logarithm, and from logarithms, all operations follow. The system achieves 100% correctness across 3,344 test cases, handles exact rational arithmetic, supports multi-operand expressions with standard precedence, and cleanly separates passive geometry from active observation.

The framework is not a simulation of arithmetic — it is arithmetic, performed through spatial relationships. The geometry does not represent numbers; it embodies them. The Observer does not decode operations; it reads them from the spatial structure.

This is computation through geometry. One primitive. All operations. Just like EML.

References

1. Adleman, L. M. (1994). Molecular computation of solutions to combinatorial problems. *Science*, 266(5187), 1021–1024.
2. Blumenthal, L. M. (1953). *Theory and Applications of Distance*

Geometry. Oxford University Press.

3. Carlsson, G. (2009). Topology and data. *BAMS*, 46(2), 255–308.
4. Connelly, R. (2005). Rigidity and flexibility. *Handbook of Discrete and Computational Geometry*, 30, 1031–1052.
5. Demaine, E. D., & O'Rourke, J. (2007). *Geometric Folding Algorithms*. Cambridge University Press.
6. Doran, C., & Lasenby, A. (2003). *Geometric Algebra for Physicists*. Cambridge University Press.
7. Edelsbrunner, H., & Harer, J. (2010). *Computational Topology*. AMS.
8. Gromov, M. (1987). Hyperbolic groups. *Essays in Group Theory*, 8, 75–263.
9. Hestenes, D. (1966). *Space-Time Algebra*. Gordon and Breach.
10. Liberti, L., et al. (2014). Euclidean distance geometry and applications. *SIAM Review*, 56(1), 3–69.
11. Odrzywolek, A. (2006). All elementary functions from a single operator. Jagiellonian University.
12. Rothmund, P. W. (2006). Folding DNA to create nanoscale shapes. *Nature*, 440, 297–302.
13. Schoenberg, I. J. (1935). Remarks to Maurice Fréchet's article. *Annals of Mathematics*, 36, 724–732.
14. Seeman, N. C. (2003). DNA nanotechnology. *Nature*, 421, 427–431.
15. Shannon, C. E. (1948). A mathematical theory of communication. *BSTJ*, 27(3), 379–423.
16. Soloveichik, D., et al. (2010). DNA as a universal substrate for chemical kinetics. *PNAS*, 107, 5393–5398.
17. von Neumann, J. (1956). Probabilistic logics. *Automata Studies*, 34, 43–98.
18. Zhang, D. Y., & Winfree, E. (2009). Control of DNA strand displacement kinetics. *JACS*, 131, 17303–17314.
19. Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274.

Spatial Arithmetic — Complete Unit

Revised: July 2026

- **Also covers:** 'tgic_v3.py', `ldp_codec.py` and a bit of the UBP 'Y' constant.

Implementation: `spatial_arithmetic.py`

1. Scope and claims

Spatial Arithmetic is a geometry-based **codec and evaluator**. Signed integers are regular unit-edge polygons embedded in 3-D; polygon vertex count stores the integer. Geometry carries values and structure, while Python performs exact `Fraction` arithmetic after decoding. Passive shapes do not calculate by themselves.

This revision implements all six requested extension areas: expression trees, oriented rational operands, calibrated recovery of complete noisy cycles, spatially indexed edge discovery, JSON/Blender interchange, and complex EML. Each has an explicit boundary described below.

2. Integer polygons

For integer v , the vertex count is

$$\begin{aligned} N(v) &= 2|v| + 4 && \text{if } v \geq 0 \\ N(v) &= 2|v| + 5 && \text{if } v < 0 \end{aligned}$$

Even counts encode non-negative values and odd counts encode negative values. The polygon is regular with unit edge and circumradius

$R(n) = 1/(2 \sin(\pi/n))$, then receives a deterministic rigid 3-D rotation. The inverse reads count and parity. Every recovered component must be one simple cycle; malformed geometry is rejected.

The flat legacy scene format places operands left-to-right. Clearance between bounding circles encodes multiply=4, divide=5, add=6, subtract=7. This remains available through `build_scene`, `build_expression`, `observe_scene`, and `observe_expression`.

3. Parentheses and expression-tree geometry

`parse_expression_tree` accepts integer literals, unary signs on literals, + - * /, and parentheses. It produces explicit binary `ExpressionNodes`. `evaluate_expression_tree` evaluates the tree with exact rational arithmetic, so grouping is preserved:

```
python3 spatial_arithmetic.py --eval "(3 + 4) * 5"
# (3 + 4) * 5 = 35
```

`expression_tree_scene` gives every node a stable path (R, RL, RR, ...). Operator records name both child paths; leaves contain their geometric rational operand. Thus topology, rather than an accidental 2-D layout, carries parentheses and survives coordinate rotation or CAD export.

Unary minus directly before a parenthesized group is deliberately rejected; write `-1*(...)`. This keeps the grammar small and explicit.

4. Unambiguous rational operands

A reduced `Fraction(p,q)` is encoded as two signed-integer polygons:

- numerator on the positive side of an orientation axis;
- denominator on the negative side;

- the stored axis points from denominator to numerator.

`decode_rational` validates both cycles, checks orientation, rejects a zero denominator, and returns a reduced `Fraction`. Swapping the polygons without also changing the orientation is an error, not a silent reciprocal.

5. Noisy scans and scalable edge discovery

`cluster_detect` now uses a uniform 3-D spatial hash. It compares points only in neighboring cells instead of making every possible pair comparison. For normally distributed sparse geometry this changes edge discovery from quadratic behavior to near-linear behavior; adversarially dense cells can still be quadratic.

`observe_noisy_operand` handles an isolated, uniformly scaled complete scan:

1. estimate edge scale by the median nearest-neighbor distance;
2. derive an absolute tolerance from a caller-supplied relative tolerance;
3. spatially recover approximately unit edges;
4. require exactly one degree-two simple cycle;
5. decode its vertex count.

The default tolerance is intentionally conservative. It recovers coordinate perturbation but **does not invent missing vertices or bridge occlusions**. Incomplete scans are rejected because guessing a missing count can silently change the represented number. Real equipment should calibrate tolerance from repeat scans and known reference artefacts.

6. JSON and Blender interchange

`save_scene_json` stores the authoritative `spatial-arithmetic-tree-v1` scene: coordinates, node paths, child links, operators, values,

and rational orientation axes. `load_scene_json` validates the format marker and basic shape.

`save_scene_obj` writes Wavefront OBJ polylines importable by Blender:

```
python3 spatial_arithmetic.py --export "2*(7-3)" scene.js
```

OBJ preserves visible polygon geometry but not arithmetic metadata, so JSON is the round-trip sidecar. In Blender use **File** → **Import** → **Wavefront (.obj)**. Measurement/CAD tools can consume the JSON directly or convert its vertex arrays to their own native format.

7. Real and complex EML

Real mode is

$$\text{eml}(x, y) = \exp(x) - \ln(y), \quad y > 0.$$

Complex mode uses the principal logarithm:

$$\text{Log}(y) = \ln|y| + i \text{Arg}(y), \quad \text{Arg}(y) \text{ in } (-\pi, \pi].$$

The branch cut is the non-positive real axis and zero is undefined. Python's `cmath.log` assigns $\text{Arg}(-r) = \pi$ for an exact negative real $-r$; limits from above and below have $+\pi$ and $-\pi$, respectively.

```
python3 spatial_arithmetic.py --eml-complex 0 -1  
# (1-3.141592653589793j)
```

EML remains a numerical primitive separate from geometric decoding. The script does not claim that ordinary arithmetic has been synthesized solely by EML compositions.

8. Exactness and verification

Decoded arithmetic uses `Fraction` and is exact. Construction, rotation, measurement, noise calibration, and EML use binary floating point. Accuracy is therefore relative to documented domains and tolerances, not every imaginable point cloud.

Run the deterministic checks with:

```
python3 spatial_arithmetic.py --self-test
```

They cover signed integer round trips, unit edges, all operators, precedence, parentheses, rational orientation, scaled noisy cycles, JSON/OBJ interchange, real and complex EML domains, centroid distance, and malformed syntax.

`RequestProject/Main.lean` independently kernel-checks exact node-count and real-EML identities. It does not formalize floating-point scanning behavior.

9. Companion-file review

- `ubp_unified_v5.py` runs its supplied 37-problem suite successfully; its own header correctly distinguishes exact arithmetic, reduced computational models, and empirical comparisons. Its companion `Main.lean` was left unchanged as requested.
- `tgic_v3.py` runs and reports 4096 codewords, 759 octads, the expected weight distribution, and a successful encode/decode check. Its randomized MOG-key hunt means projections are not reproducible across processes unless the random generator is controlled; reported physical interpretations remain model claims rather than consequences of these finite code checks.
- `ldp_codec.py` grouped integers correctly, but its old bit accounting called an index into an unstored member table “compression” while retaining all original values. The revision

keeps its API but labels it grouping analysis, validates ranges/types, and includes exact range codes in its standalone storage estimate. Consequently its honest “savings” may be negative.

- `Y.txt` contains several valid algebraic observations, including $Y = \pi / (\pi^2 + 2)$ and $1/Y = \pi + 2/\pi$, but its numerical search, closed-loop, entropy, and physical statements are not reproducible from the document alone. Also, $\pi + 2/\pi$ is a **sum**, not the arithmetic mean (the latter is half that value). Claims about uniqueness are only relative to the tested candidate list, and physical significance requires independent experimental evidence.

These reviews do not couple the companion studies into Spatial Arithmetic: the codec stays a small standalone standard-library module.